
Security Review Report
NM-0997 - ANOMA - XAN TOKEN-UPGRADE



NETHERMIND
SECURITY

(August 6, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Governance	4
5	Risk Rating Methodology	5
6	Issues	6
6.1	[Info] The council can schedule a token upgrade that the voter body is unable to cancel	6
7	Documentation Evaluation	7
8	Test Suite Evaluation	8
8.1	Tests Output	8
9	About Nethermind	17

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for [Anoma's](#) XAN token and its governance layer. The primary focus of the engagement has been the V1 → V2 upgrade of the live XAN token and the external governance stack that assumes ownership of it.

XAN is a fixed-supply ERC-20 behind a UUPS proxy whose first implementation governed itself, gating upgrades through an in-token quorum vote weighted by locked balances. **V2** replaces that machinery with a single Ownable owner, releases the formerly locked balances on a three-year straight-line schedule, and adds ERC20Votes delegation on a timestamp clock so that governance can be conducted about the token from outside it. Ownership passes to an [OpenZeppelin](#) Governor and TimelockController, which together impose a mandatory delay on every privileged action and leave execution open to any caller. Furthermore, the protocol incorporates a XanUpgradeCouncilModule which is able to initiate a token upgrade as a liveness fallback should the voter body be unable to reach quorum, on a deliberately longer timeline than a voter proposal so that the voter body retains the ability to cancel any upgrade the council schedules.

The audit comprises approximately 465 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 1 point of attention, where the issue is classified as Informational. The issue is summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

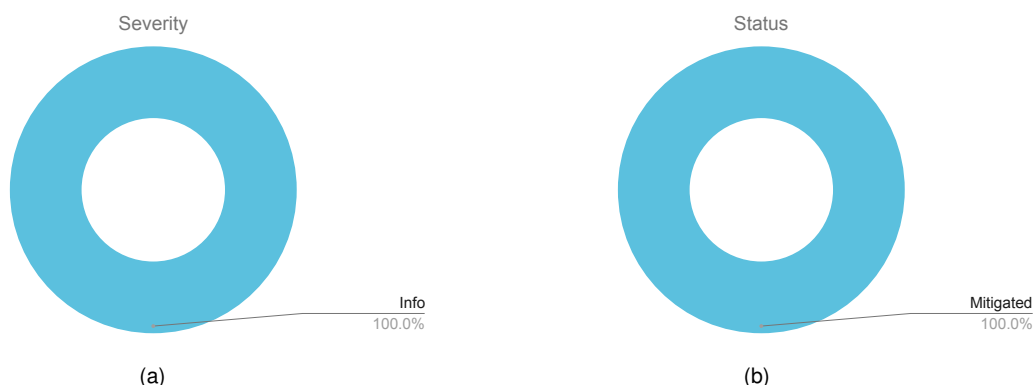


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (0), Undetermined (0), Informational (1), Best Practices (0).
Distribution of status: Fixed (0), Acknowledged (0), Mitigated (1), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	Aug 5, 2026
Final Report	Aug 6, 2026
Initial Commit	Final Commit
6ebf646	
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/XanV2.sol	169	119	70%	60	348
2	src/XanGovernor.sol	118	46	39%	17	181
3	src/XanUpgradeCouncilModule.sol	92	55	59%	37	184
4	script/PrepareXanV2Upgrade.s.sol	66	37	56%	15	118
5	script/ExecuteXanV2Upgrade.s.sol	20	5	25%	11	36
	Total	465	262	56.3%	140	867

3 Summary of Issues

	Finding	Severity	Update
1	The council can schedule a token upgrade that the voter body is unable to cancel	Info	Mitigated

4 System Overview

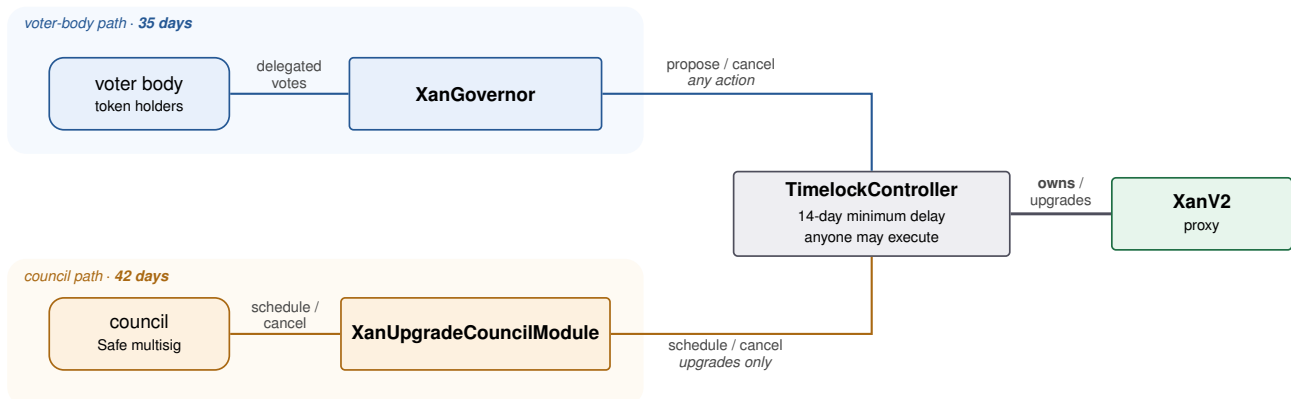
XAN is the token of the Anoma network: a **fixed-supply ERC-20** behind a UUPS proxy, with ten billion tokens minted once at initialisation and no mechanism to mint or burn thereafter.

Under V1 the token *governed itself*. Upgrades were gated by machinery built into the token: holders locked tokens, voted for a candidate implementation, and once a quorum of the locked supply backed one it could be scheduled and executed after a delay, with a council multisig able to skip the quorum-accumulation phase. Locking existed to serve that vote — voting weight *was* the locked balance — and V1 offered no way to release those tokens once locked.

XAN Token V2 dismantles that arrangement through three changes:

- In-token governance is removed** and replaced by a single Ownable owner, which becomes the sole authority over upgrades. The token no longer knows what a vote is.
- The locked balances are released** on a three-year straight-line schedule. The locking mechanism they served no longer exists, so the locks are discharged rather than left stranded.
- ERC20Votes delegation is added**, on a timestamp clock, so that governance can be conducted *about* the token from outside it.

The result is a division of concerns. The token becomes a **passive asset with exactly one privileged operation** — no pause, mint, burn, blacklist, fee switch, or admin rescue exists — while the owner role passes to an external stack: an OpenZeppelin Governor driven by the token's votes, a TimelockController holding the owner role, and a council module able to initiate upgrades as a liveness fallback. Because that stack sits outside the token, ownership can be transferred to an entirely different governance system with no token upgrade at all.



Authority flows left to right. Both paths converge on the timelock, which holds `owner()` and is the only address able to upgrade the token.

4.1 Governance

Governance sits entirely outside the token, and its only subject is the token's single privileged operation: who may upgrade it, and after how long.

The timelock is the pivot. It holds `owner()`, is the only address that can upgrade the token, and imposes a 14-day minimum on every operation. Two configuration choices shape everything above it. Execution is **permissionless** — `EXECUTOR_ROLE` is held by `address(0)` — so no party can hold a passed decision hostage by refusing to act, and authority therefore reduces entirely to *who may schedule and who may cancel*. And the timelock **administers itself**, the deployer's transient admin having been renounced at deployment, so the role set changes only through a passed proposal.

The voter body schedules through XanGovernor, a stock OpenZeppelin governor with no bespoke powers. Its 7-day pre-voting delay is where the vote-weight snapshot is taken, so tokens cannot be acquired and voted with in the same breath. Passing requires `For + Abstain` to reach quorum — a fixed 1,000,000,000 XAN, being 10% of supply — and `For > Against`; only *delegated* tokens count toward either.

The council schedules through XanUpgradeCouncilModule, which is non-upgradeable, all-immutable, and never owns the token. It exists for one scenario: the voter body cannot reach quorum and the token still needs upgrading. Granting a multisig that power is made safe by **timing rather than trust** — `upgradeDelay()` is computed live as `votingDelay + votingPeriod + minDelay + COUNCIL_EXTRA_DELAY`, a full voter cycle plus a week, so the voters always have room to cancel whatever it schedules. The module exposes two mutating functions:

- `scheduleUpgrade(newImplementation, data)`** — the target is the immutable token address and the calldata shape is fixed to `upgradeToAndCall`, so it cannot grant a role, move funds, or call the governor. One upgrade in flight at a time.
- `cancelUpgrade()`** — takes **no arguments**; the operation id comes from storage the module wrote itself, which is what makes holding a cancellation role safe, since there is no parameter through which it could be aimed at a voter-body operation.

The council address is immutable. Safe membership changes need no on-chain action, while replacing the address means deploying a fresh module and re-wiring roles through voter proposals — the same mechanism by which the voter body disarms a captured council.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Info] The council can schedule a token upgrade that the voter body is unable to cancel

File(s): [XanUpgradeCouncilModule.sol](#)

Description: The council may replace the token's entire implementation. That power is considered safe only because the voter body can always cancel a scheduled upgrade before it executes. With the current parameters a council upgrade waits **42 days** while a cancellation proposal takes **35 days**, leaving a 7-day margin to notice and begin. That margin can be removed, leaving the council with an upgrade nobody can stop. The following four behaviors each reasonable alone but combined, enable this issue:

- The upgrade deadline is stored as a date, never recomputed.** `upgradeDelay()` is evaluated once at ; scheduling and the timelock keeps only the resulting timestamp — it holds no reference to the governor and never consults it again:

```
// TimelockController.sol:322
_timestamps[id] = block.timestamp + delay;
```

- Anyone may execute a passed proposal, so the council can execute one the voter body passed.** `Governor.execute` carries no access control (`Governor.sol:390`), so any address may execute a proposal in the Succeeded or Queued state. Its relay to the timelock succeeds because `EXECUTOR_ROLE` is held by `address(0)`, which I documented as intentional — *"anyone may execute once the delay elapses"* — so that no party can block an approved decision by refusing to act. No change to either is proposed.
- The council controls the ordering.** A Safe bundles several calls into one transaction, so it decides which happens first.
- A cancellation reads the settings live**, at the moment it is created:

```
// Governor.sol:320-321
uint256 snapshot = clock() + votingDelay();
uint256 duration = votingPeriod();
```

The precondition is a proposal the voter body has already passed that lengthens the voting period by at least 7-day margin — say from 14 to 28 days — which is queued in the timelock and past its delay, but not yet executed. Ripe-but-unexecuted is an ordinary state, since nobody is obliged to execute promptly.

The council then submits a single transaction:

	Call	Effect
1	<code>module.scheduleUpgrade(maliciousImpl, data)</code>	<code>upgradeDelay()</code> reads <code>votingPeriod</code> as 14 , giving 42 days. Upgrade executable on day 42 — now a fixed date
2	<code>governor.execute(...)</code> on the queued proposal	<code>votingPeriod</code> becomes 28

Call 2 must go through the governor rather than the timelock directly: `setVotingPeriod` is onlyGovernance, which requires both that the caller is the timelock **and** that the calldata was registered in `_governanceCall`, which happens only inside `Governor.execute`. Calling `timelock.executeBatch(...)` directly would revert on an empty queue.

From that point every cancellation proposal costs $7 + 28 + 14 = 49$ days, so the earliest possible completion is day 49 while the upgrade executes on day 42. Because both calls are atomic, no cancellation can be filed between them: any earlier transaction has no operation to cancel yet, and any later one is subject to the 28-day period. The voter body can reject the upgrade unanimously and still be seven days late.

The order is what makes it work. Reversed, `upgradeDelay()` would read 28 and produce a 56-day deadline, leaving the voters ample room. Batching guarantees that order; separate transactions would also work in practice, though they would leave a window in which a cancellation could in principle be filed first.

No alternative route is faster: revoking the module's timelock roles or transferring token ownership away from the timelock both require proposals of their own, and only the council may call `cancelUpgrade()`.

What limits it. The council cannot create the precondition — the voter body must have passed the increase and left it queued. An increase of 7 days or less only shortens the margin rather than removing it, and one executed more than 7 days after scheduling is irrelevant, because by then the margin had already expired on ordinary grounds.

Recommendation(s): Consider making the guarantee unconditional, either by capping how far `setVotingDelay/setVotingPeriod` can be raised and sizing `COUNCIL_EXTRA_DELAY` to cover that cap, or by applying a fixed floor to the scheduled delay

Status: Mitigated [here](#)

Update from Nethermind: The [mitigation](#) still allows a scenario where two contrarian proposals, scheduled but not executed where the first one increases either the `votingPeriod` or `votingDelay` and the second one reduces it opens up an opportunity for the vulnerability to still exist. Although the likelihood for this precondition to enable exploitation is very low.

Update from the client: Acknowledged and documentation updated [here](#)

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Anoma's documentation

The Anoma team provided a comprehensive overview of the main system components during the kick-off call with a detailed explanation of the intended functionalities along with a written specification for the different contracts and roles. Additionally, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and in depth understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

```

forge test --force
[] Compiling...
[] Compiling 156 files with Solc 0.8.36
[] Solc 0.8.36 finished in 52.62s
Compiler run successful!

Ran 1 test for test/XanV2.storage.t.sol:XanV2StorageTest
[PASS] test_storage_slot() (gas: 1289)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 238.08µs (18.88µs CPU time)

Ran 3 tests for test/XanGovernor.upgrade.t.sol:XanGovernorUpgradeTest
[PASS] test_dao_upgrades_the_token_to_v3() (gas: 3191096)
[PASS] test_direct_upgrade_by_a_non_owner_reverts() (gas: 2948431)
[PASS] test_voter_votes_equal_balance() (gas: 44419)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 7.09s (332.13µs CPU time)

Ran 10 tests for test/XanGovernor.voting.t.sol:XanGovernorVotingTest
[PASS] test_abstain_alone_does_not_pass_a_proposal() (gas: 143319)
[PASS] test_abstain_counts_toward_quorum() (gas: 208967)
[PASS] test_passing_proposal_executes_a_treasury_transfer() (gas: 403043)
[PASS] test_proposal_is_defeated_when_for_ties_against() (gas: 313057)
[PASS] test_proposal_is_defeated_when_quorum_is_not_reached() (gas: 147047)
[PASS] test_proposal_is_defeated_without_quorum() (gas: 89806)
[PASS] test_proposal_succeeds_when_for_outweighs_against() (gas: 240456)
[PASS] test_proposal_succeeds_when_quorum_is_exactly_met() (gas: 189181)
[PASS] test_votes_are_tallied_by_delegated_weight() (gas: 244586)
[PASS] test_voting_power_reflects_the_distribution() (gas: 59261)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 9.76s (925.04µs CPU time)

Ran 3 tests for test/XanV1.initialization.t.sol:XanV1InitializationTest
[PASS] test_implementation_points_to_the_correct_implementation() (gas: 3430087)
[PASS] test_initialize_mints_the_expected_supply_amounting_to_10_billion_tokens() (gas: 13533)
[PASS] test_initialize_mints_the_supply_for_the_specified_owner() (gas: 20634)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.68s (115.29µs CPU time)

Ran 4 tests for test/XanV2.upgrade.t.sol:XanV2UpgradeTest
[PASS] test_implementation_returns_the_current_implementation() (gas: 13129)
[PASS] test_upgradeToAndCall_reverts_if_the_caller_is_not_the_owner() (gas: 2878111)
[PASS] test_upgradeToAndCall_reverts_if_the_new_implementation_is_the_V1_implementation() (gas: 20909)
[PASS] test_upgradeToAndCall_upgrades_if_the_caller_is_the_owner() (gas: 2889110)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 1.98s (187.63µs CPU time)

Ran 15 tests for test/XanV2.voting.t.sol:XanV2VotingTest
[PASS] test_CLOCK_MODE_returns_the_timestamp_mode() (gas: 14605)
[PASS] test_clock_tracks_the_block_timestamp() (gas: 11002)
[PASS] test_delegate_grants_voting_power_equal_to_balance() (gas: 95486)
[PASS] test_getPastTotalSupply_is_zero_before_the_upgrade() (gas: 18034)
[PASS] test_getPastTotalSupply_returns_the_seeded_supply_after_upgrade() (gas: 21509)
[PASS] test_getPastTotalSupply_reverts_during_the_upgrade() (gas: 17087)
[PASS] test_getPastVotes_returns_the_checkpointed_value() (gas: 92759)
[PASS] test_getVotes_counts_fully_unlocked_tokens_after_vesting() (gas: 130082)
[PASS] test_getVotes_counts_locked_tokens_before_vesting_starts() (gas: 102958)
[PASS] test_getVotes_counts_partially_unlocked_tokens_during_vesting() (gas: 130499)
[PASS] test_getVotes_counts_vested_but_not_unlocked_tokens_after_vesting() (gas: 105148)
[PASS] test_getVotes_counts_vested_but_not_unlocked_tokens_during_vesting() (gas: 105719)
[PASS] test_getVotes_is_unchanged_by_unlocking_across_vesting() (gas: 134488)
[PASS] test_getVotes_returns_zero_before_delegation() (gas: 15827)
[PASS] test_transfer_moves_voting_power_between_delegates() (gas: 236652)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 1.82s (377.88µs CPU time)

Ran 11 tests for test/script/PrepareXanV2Upgrade.t.sol:PrepareXanV2UpgradeTest
[PASS] testFuzz_grantRole_reverts_if_the_caller_is_not_the_timelock(address) (runs: 1000, : 16384, ~: 16384)
[PASS] test_deployGovernance_configures_the_governor() (gas: 26987)
[PASS] test_deployGovernance_grants_scheduling_and_cancelling_to_the_governor_and_the_upgrade_council() (gas: 36583)
[PASS] test_deployGovernance_leaves_the_timelock_self_administered() (gas: 28970)
[PASS] test_deployGovernance_opens_the_executor_role() (gas: 8889)
[PASS] test_deployGovernance_reverts_if_the_council_is_the_zero_address() (gas: 11182)
[PASS] test_deployGovernance_reverts_if_the_token_is_the_zero_address() (gas: 9296)

```

```
[PASS] test_deployGovernance_sets_the_timelock_min_delay() (gas: 8154)
[PASS] test_deployGovernance_wires_the_upgrade_council() (gas: 21489)
[PASS] test_quorum_numerator_is_checkpointed_at_the_deploy_timestamp() (gas: 12712)
[PASS] test_run_deploys_governance_and_prepares_the_upgrade() (gas: 32351381)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 16.50s (3.49s CPU time)
```

```
Ran 5 tests for test/XanGovernor.proposal.t.sol:XanGovernorProposalTest
[PASS] test_cancel_cancels_pending_proposals_if_called_by_the_proposer() (gas: 84460)
[PASS] test_relay_reverts_for_pending_proposals_if_the_caller_is_not_the_proposer() (gas: 78483)
[PASS] test_proposalNeedsQueuing_returns_true_for_proposals() (gas: 70630)
[PASS] test_proposalThreshold_gates_proposal_creation() (gas: 30941)
[PASS] test_quorum_is_a_tenth_of_the_voting_supply() (gas: 24459)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 1.67s (237.04µs CPU time)
```

```
Ran 4 tests for test/XanGovernor.security.t.sol:XanGovernorSecurityTest
[PASS] test_quorum_changes_only_through_a_passed_proposal() (gas: 299987)
[PASS] test_relay_reverts_if_not_called_through_governance() (gas: 16929)
[PASS] test_settings_setters_revert_if_not_called_through_governance() (gas: 28659)
[PASS] test_timelock_updateDelay_reverts_if_not_called_by_the_timelock() (gas: 11878)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.02s (231.29µs CPU time)
```

```
Ran 2 tests for test/XanGovernor.unit.t.sol:XanGovernorUnitTest
[PASS] test_CLOCK_MODE_is_timestamp() (gas: 9630)
[PASS] test_clock_returns_the_timestamp() (gas: 6291)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 808.25µs (20.83µs CPU time)
```

```
Ran 17 tests for test/XanV1.locking.t.sol:XanV1LockingTest
[PASS] invariant_lockedBalance() (runs: 500, calls: 32000, reverts: 29882)
```

Contract	Selector	Calls	Reverts	Discards
XanV1	approve	1988	16	0
XanV1	burn	1966	1941	0
XanV1	burnFrom	1979	1953	0
XanV1	cancelCouncilUpgrade	2000	2000	0
XanV1	cancelVoterBodyUpgrade	1953	1953	0
XanV1	castVote	1970	1970	0
XanV1	initializeV1	2056	2056	0
XanV1	lock	2061	2032	0
XanV1	permit	2027	2027	0
XanV1	scheduleCouncilUpgrade	1960	1960	0
XanV1	scheduleVoterBodyUpgrade	1951	1951	0
XanV1	transfer	2034	2007	0
XanV1	transferAndLock	1989	1989	0
XanV1	transferFrom	1954	1915	0
XanV1	upgradeToAndCall	2010	2010	0
XanV1	vetoCouncilUpgrade	2102	2102	0

```
[PASS] testFuzz_lockedBalanceOf_and_unlockedBalanceOf_sum_to_balanceOf(address) (runs: 1000, : 18822, ~: 18822)
[PASS] test_burn_burns_unlocked_tokens() (gas: 24832)
[PASS] test_burn_reverts_on_insufficient_unlocked_tokens() (gas: 69622)
[PASS] test_lock_emits_the_Locked_event() (gas: 69838)
[PASS] test_lock_locks_incrementally() (gas: 75161)
[PASS] test_lockedBalanceOf_is_bound_by_balanceOf(address) (runs: 1000, : 16564, ~: 16564)
[PASS] test_lockedBalanceOf_returns_the_locked_balance() (gas: 69922)
[PASS] test_lockedSupply_returns_the_locked_supply() (gas: 82830)
[PASS] test_transferAndLock_reverts_if_the_caller_is_not_authorized(address) (runs: 1000, : 57544, ~: 57544)
[PASS] test_transferAndLock_reverts_on_insufficient_unlocked_tokens() (gas: 76088)
```

```
[PASS] test_transfer_emits_the_Transfer_event() (gas: 50449)
[PASS] test_transfer_reverts_on_insufficient_unlocked_tokens() (gas: 73487)
[PASS] test_transfer_transfers_and_locks_tokens() (gas: 120313)
[PASS] test_transfer_transfers_tokens() (gas: 73534)
[PASS] test_unlockedBalanceOf_is_bound_by_balanceOf(address) (runs: 1000, : 17002, ~: 17002)
[PASS] test_unlockedBalanceOf_returns_the_unlocked_balance() (gas: 71621)
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 20.25s (523.59ms CPU time)
```

```
Ran 2 tests for test/XanV1.permit.t.sol:XanV1PermitTest
[PASS] test_permits_reverts_for_invalid_signature() (gas: 45830)
[PASS] test_permits_spending_given_an_EIP712_signature() (gas: 79021)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.73s (587.33µs CPU time)
```

```
Ran 1 test for test/XanV1.storage.t.sol:XanV1StorageTest
[PASS] test_storage_slot() (gas: 1135)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 192.00µs (8.50µs CPU time)
```

```
Ran 18 tests for test/XanV1.upgrade.t.sol:XanV1UpgradeTest
[PASS] invariant_mutually_exclusive_schedule_upgrades() (runs: 500, calls: 32000, reverts: 27405)
```

Contract	Selector	Calls	Reverts	Discards
XanV1	approve	647	2	0
XanV1	burn	660	650	0
XanV1	burnFrom	707	700	0
XanV1	cancelCouncilUpgrade	657	657	0
XanV1	cancelVoterBodyUpgrade	669	669	0
XanV1	castVote	609	609	0
XanV1	initializeV1	637	637	0
XanV1	lock	682	675	0
XanV1	permit	643	643	0
XanV1	scheduleCouncilUpgrade	668	668	0
XanV1	scheduleVoterBodyUpgrade	639	639	0
XanV1	transfer	653	644	0
XanV1	transferAndLock	624	624	0
XanV1	transferFrom	641	637	0
XanV1	upgradeToAndCall	698	698	0
XanV1	vetoCouncilUpgrade	653	653	0
XanV2	approve	1955	20	0
XanV2	delegate	1921	0	0
XanV2	delegateBySig	1929	1929	0
XanV2	permit	1971	1971	0
XanV2	reinitializeFromV1	1987	1987	0
XanV2	renounceOwnership	1896	1896	0
XanV2	transfer	1961	1933	0
XanV2	transferFrom	2024	1995	0
XanV2	transferOwnership	1996	1996	0
XanV2	unlock	1954	1954	0

```
| XanV2 | upgradeToAndCall | 1919 | 1919 | 0 |
-----+-----+-----+-----+-----+
[PASS] test_authorizeUpgrade_passes_council_upgrade_if_the_voter_body_implementation_is_below_quorum() (gas: 174467)
[PASS] test_authorizeUpgrade_passes_if_the_voter_body_has_scheduled_the_upgrade_after_the_council() (gas: 181095)
[PASS] test_authorizeUpgrade_passes_if_the_voter_body_has_scheduled_the_upgrade_in_the_same_block_as_the_council()
↳ (gas: 180444)
[PASS] test_authorizeUpgrade_resets_the_scheduled_council_upgrade_data_on_upgrade_execution() (gas: 46178)
[PASS] test_authorizeUpgrade_resets_the_scheduled_voter_body_upgrade_data_on_upgrade_execution() (gas: 164473)
[PASS] test_authorizeUpgrade_reverts_council_upgrade_if_the_delay_period_has_not_ended() (gas: 58022)
[PASS] test_authorizeUpgrade_reverts_council_upgrade_if_the_voter_body_implementation_has_reached_quorum() (gas:
↳ 184158)
[PASS] test_authorizeUpgrade_reverts_for_an_upgrade_to_address_0() (gas: 15051)
[PASS] test_authorizeUpgrade_reverts_voter_body_upgrade_if_delay_period_has_not_ended() (gas: 176604)
[PASS] test_authorizeUpgrade_reverts_voter_body_upgrade_if_delay_period_has_not_started() (gas: 151729)
[PASS] test_authorizeUpgrade_reverts_voter_body_upgrade_if_implementation_has_not_been_scheduled() (gas: 151368)
[PASS] test_authorizeUpgrade_reverts_voter_body_upgrade_if_implementation_is_not_the_most_voted() (gas: 237919)
[PASS] test_authorizeUpgrade_reverts_voter_body_upgrade_if_no_implementation_has_been_scheduled() (gas: 23979)
[PASS] test_authorizeUpgrade_reverts_voter_body_upgrade_if_quorum_is_lost_after_scheduling() (gas: 180881)
[PASS] test_authorizeUpgrade_reverts_voter_body_upgrade_if_the_delay_period_has_passed_for_a_different_implementation()
↳ (gas: 178139)
[PASS] test_upgradeToAndCall_allows_upgrade_to_the_current_implementation() (gas: 46452)
[PASS] test_upgradeToAndCall_emits_the_Upgrade_event() (gas: 135209)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in 8.19s (540.64ms CPU time)

Ran 1 test for test/XanV2.upgrade.integration.t.sol:XanV2UpgradeIntegrationTest
[PASS] test_scripts_drive_the_full_v1_to_v2_upgrade() (gas: 81408638)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.98s (3.98s CPU time)

Ran 19 tests for test/XanV1.council.t.sol:XanV1CouncilTest
[PASS] test_cancelCouncilUpgrade_cancels_the_council_upgrade_after_the_delay_has_passed() (gas: 41390)
[PASS] test_cancelCouncilUpgrade_cancels_the_council_upgrade_before_the_delay_has_passed() (gas: 40840)
[PASS] test_cancelCouncilUpgrade_emits_the_CouncilUpgradeCancelled_event() (gas: 38712)
[PASS] test_cancelCouncilUpgrade_resets_the_scheduled_upgrade_to_zero() (gas: 39207)
[PASS] test_cancelCouncilUpgrade_reverts_if_no_council_upgrade_is_scheduled() (gas: 19529)
[PASS] test_cancelCouncilUpgrade_reverts_if_the_caller_is_not_the_council() (gas: 53375)
[PASS] test_scheduleCouncilUpgrade_emits_the_CouncilUpgradeScheduled_event() (gas: 50673)
[PASS] test_scheduleCouncilUpgrade_proposes_an_upgrade() (gas: 48156)
[PASS] test_scheduleCouncilUpgrade_proposes_an_upgrade_to_the_same_implementation() (gas: 48018)
[PASS] test_scheduleCouncilUpgrade_reverts_if_an_council_upgrade_has_been_proposed_already() (gas: 53491)
[PASS]
↳ test_scheduleCouncilUpgrade_reverts_if_an_voter_body_upgrade_has_quorum_and_the_minimal_locked_supply_is_reached()
↳ (gas: 178863)
[PASS]
↳ test_scheduleCouncilUpgrade_reverts_if_an_voter_body_upgrade_has_quorum_and_the_minimal_locked_supply_is_reached-
↳ _even_if_it_is_the_same_implementation() (gas: 178378)
[PASS] test_scheduleCouncilUpgrade_reverts_if_the_caller_is_not_the_council() (gas: 20063)
[PASS] test_scheduledCouncilImplementation_returns_0_if_no_upgrade_delay_has_been_started() (gas: 13206)
[PASS] test_scheduledCouncilImplementation_returns_the_scheduled_upgrade_if_an_upgrade_has_been_scheduled() (gas:
↳ 50464)
[PASS] test_vetoCouncilUpgrade_emits_the_CouncilUpgradeVetoed_event() (gas: 156584)
[PASS] test_vetoCouncilUpgrade_reverts_if_no_implementation_proposed_by_the_voter_body_has_reached_quorum() (gas:
↳ 56636)
[PASS] test_vetoCouncilUpgrade_vetos_the_council_upgrade_after_the_delay_has_passed() (gas: 159552)
[PASS] test_vetoCouncilUpgrade_vetos_the_council_upgrade_before_the_delay_has_passed() (gas: 158444)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 24.79s (542.79µs CPU time)

Ran 25 tests for test/XanV1.voting.t.sol:XanV1VotingTest
[PASS]
↳ test_cancelVoterBodyUpgrade_cancels_a_scheduled_upgrade_after_the_delay_if_the_implementation_is_not_the_most_voted-
↳ _anymore() (gas: 214087)
[PASS] test_cancelVoterBodyUpgrade_emits_the_DelayReset_event() (gas: 213728)
[PASS] test_cancelVoterBodyUpgrade_resets_the_delay() (gas: 216858)
[PASS]
↳ test_cancelVoterBodyUpgrade_reverts_after_the_delay_has_passed_when_attempting_to_cancel_the_most_voted_implemen-
↳ tion() (gas: 174424)
[PASS] test_cancelVoterBodyUpgrade_reverts_if_no_upgrade_is_scheduled() (gas: 16579)
[PASS] test_cancelVoterBodyUpgrade_reverts_if_the_delay_period_has_not_ended() (gas: 172407)
[PASS]
↳ test_castVote_does_not_update_the_most_voted_implementation_if_the_proposed_implementation_has_the_same_number_of-
↳ _votes() (gas: 192496)
[PASS] test_castVote_emits_the_VoteCast_event() (gas: 145136)
[PASS] test_castVote_increases_votes_if_more_tokens_have_been_locked() (gas: 157109)
[PASS] test_castVote_reverts_if_caller_has_no_locked_tokens() (gas: 21624)
[PASS] test_castVote_reverts_if_the_votes_has_already_been_casted() (gas: 145523)
```

```
[PASS] test_castVote_reverts_if_zero_tokens_have_been_locked() (gas: 21602)
[PASS] test_castVote_sets_the_votes_of_the_caller_to_the_locked_balance() (gas: 160339)
[PASS] test_castVote_updates_the_most_voted_implementation_if_the_proposed_implementation_has_more_votes() (gas:
  ↳ 198016)
[PASS] test_scheduleVoterBodyUpgrade_cancels_a_scheduled_upgrade_by_the_council_after_the_delay_has_passed() (gas:
  ↳ 189082)
[PASS]
  ↳ test_scheduleVoterBodyUpgrade_cancels_a_scheduled_upgrade_by_the_council_and_resets_the_scheduled_upgrade_to_zero()
  ↳ (gas: 186345)
[PASS] test_scheduleVoterBodyUpgrade_cancels_a_scheduled_upgrade_by_the_council_before_the_delay_has_passed() (gas:
  ↳ 188160)
[PASS] test_scheduleVoterBodyUpgrade_emits_the_DelayStarted_event() (gas: 174567)
[PASS] test_scheduleVoterBodyUpgrade_returns_the_upgrade_if_one_has_been_scheduled() (gas: 171821)
[PASS] test_scheduleVoterBodyUpgrade_reverts_if_delay_has_already_been_started() (gas: 172733)
[PASS] test_scheduleVoterBodyUpgrade_reverts_if_quorum_is_not_met() (gas: 153536)
[PASS] test_scheduleVoterBodyUpgrade_reverts_if_the_minimal_locked_supply_is_not_met() (gas: 147912)
[PASS] test_scheduleVoterBodyUpgrade_schedules_the_most_voted_implementation() (gas: 172023)
[PASS]
  ↳ test_scheduleVoterBodyUpgrade_starts_the_delay_if_locked_supply_and_quorum_are_met_and_the_impl_is_the_most_voted()
  ↳ (gas: 179362)
[PASS] test_scheduledVoterBodyImplementation_returns_0_if_no_upgrade_has_been_scheduled() (gas: 13668)
Suite result: ok. 25 passed; 0 failed; 0 skipped; finished in 1.86s (848.96µs CPU time)

Ran 10 tests for test/XanV2.constructor.t.sol:XanV2ConstructorTest
[PASS] test_constructor_binds_the_initial_owner() (gas: 194407)
[PASS] test_constructor_disables_initializers_on_the_implementation() (gas: 11206)
[PASS] test_constructor_reverts_if_the_owner_is_the_zero_address() (gas: 41085)
[PASS] test_constructor_reverts_if_the_vesting_duration_is_the_zero_duration() (gas: 40876)
[PASS] test_constructor_reverts_if_the_vesting_start_is_the_zero_timestamp() (gas: 40917)
[PASS] test_constructor_sets_initialized_to_the_maximal_value() (gas: 5713)
[PASS] test_constructor_sets_the_initial_owner() (gas: 5600)
[PASS] test_constructor_sets_the_vesting_end() (gas: 6383)
[PASS] test_constructor_sets_the_vesting_start() (gas: 5786)
[PASS] test_initialOwner_does_not_track_ownership_transfers() (gas: 206108)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 478.50µs (228.96µs CPU time)

Ran 5 tests for test/XanV1.criteria.t.sol:MockXanV1CriteriaTest
[PASS] test_checkDelayCriterion_reverts_if_the_delay_period_has_not_started() (gas: 14004)
[PASS] test_isQuorumAndMinLockedSupplyReached_returns_false_if_the_min_locked_supply_is_not_met_by_one_vote() (gas:
  ↳ 143142)
[PASS] test_isQuorumAndMinLockedSupplyReached_returns_false_if_the_quorum_threshold_is_not_exceeded() (gas: 150014)
[PASS] test_isQuorumAndMinLockedSupplyReached_returns_true_if_the_min_locked_supply_is_exactly_met() (gas: 143003)
[PASS] test_isQuorumAndMinLockedSupplyReached_returns_true_if_the_quorum_threshold_is_exceeded_by_one_vote() (gas:
  ↳ 150297)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 1.71s (140.79µs CPU time)

Ran 3 tests for test/XanV2.nonces.t.sol:XanV2NoncesTest
[PASS] test_nonces_cannot_be_reused_across_signature_types() (gas: 85284)
[PASS] test_nonces_of_delegateBySig_do_not_conflict_with_nonces_of_permit() (gas: 117778)
[PASS] test_nonces_of_permit_do_not_conflict_with_nonces_of_delegateBySig() (gas: 119340)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.67s (759.00µs CPU time)

Ran 2 tests for test/script/ExecuteXanV2Upgrade.t.sol:ExecuteXanV2UpgradeTest
[PASS] test_run_reverts_before_the_delay_has_elapsed() (gas: 59461)
[PASS] test_run_reverts_if_no_upgrade_is_scheduled() (gas: 21369)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 29.80s (117.33µs CPU time)

Ran 5 tests for test/XanV1.erc20.t.sol:XanV1ERC20Test
[PASS] test_update_BURN_reverts_if_the_from_address_has_unsufficient_unlocked_tokens() (gas: 21251)
[PASS] test_update_BURN_updates_if_the_from_address_has_sufficient_unlocked_tokens() (gas: 38844)
[PASS] test_update_MINT_does_not_require_address_0_to_hold_unlocked_tokens() (gas: 58674)
[PASS] test_update_TRANSFER_reverts_transfer_if_the_from_address_has_insufficient_unlocked_tokens() (gas: 26937)
[PASS] test_update_TRANSFER_updates_if_the_from_address_has_sufficient_unlocked_tokens() (gas: 56276)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 30.52s (339.63µs CPU time)

Ran 13 tests for test/XanV2.unlocking.t.sol:XanV2UnlockingTest
[PASS] testFuzz_lockedBalanceOf_returns_the_locked_balance(uint48) (runs: 1000, : 51706, ~: 51779)
[PASS] testFuzz_principalOf_is_unchanged_by_unlocking_and_transfers(uint48) (runs: 1000, : 88186, ~: 88265)
[PASS] testFuzz_transfer_reverts_if_value_exceeds_unlocked_balance(uint48) (runs: 1000, : 52734, ~: 52810)
[PASS] testFuzz_unlockableBalanceOf_returns_the_unlockable_balance(uint48) (runs: 1000, : 48719, ~: 48793)
[PASS] testFuzz_unlockedBalanceOf_returns_the_unlocked_balance(uint48) (runs: 1000, : 50137, ~: 50212)
[PASS] test_principalOf_returns_the_formerly_locked_v1_balance() (gas: 18855)
[PASS] test_unlock_emits_the_Unlocked_event() (gas: 47276)
[PASS] test_unlock_makes_vested_tokens_transferable() (gas: 83520)
[PASS] test_unlock_returns_the_vested_balance() (gas: 45123)
```



```
[PASS] test_unlock_reverts_if_no_new_amount_has_vested() (gas: 49241)
[PASS] test_unlock_reverts_when_nothing_has_vested() (gas: 22409)
[PASS] test_vestingEnd_returns_expected_parameter() (gas: 13218)
[PASS] test_vestingStart_returns_expected_parameter() (gas: 12483)
Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 31.41s (166.07ms CPU time)
```

```
Ran 35 tests for test/XanUpgradeCouncilModule.t.sol:XanUpgradeCouncilModuleTest
[PASS] test_cancelUpgrade_lets_the_council_withdraw_its_own_upgrade() (gas: 2937749)
[PASS] test_cancelUpgrade_only_cancels_the_council_upgrade() (gas: 6028480)
[PASS] test_cancelUpgrade_reverts_if_no_upgrade_was_scheduled() (gas: 11221)
[PASS] test_cancelUpgrade_reverts_if_the_caller_is_not_the_council() (gas: 2947463)
[PASS] test_cancelUpgrade_reverts_if_the_upgrade_is_no_longer_pending() (gas: 2937702)
[PASS] test_cancelUpgrade_reverts_if_the_upgrade_was_already_executed() (gas: 2980726)
[PASS] test_constructor_reverts_if_the_council_is_the_zero_address() (gas: 45171)
[PASS] test_constructor_reverts_if_the_extra_delay_is_zero() (gas: 45135)
[PASS] test_constructor_reverts_if_the_governor_is_the_zero_address() (gas: 42949)
[PASS] test_constructor_reverts_if_the_timelock_is_the_zero_address() (gas: 42459)
[PASS] test_constructor_reverts_if_the_token_is_the_zero_address() (gas: 42483)
[PASS] test_constructor_sets_the_council() (gas: 6452)
[PASS] test_constructor_sets_the_extra_delay() (gas: 6010)
[PASS] test_constructor_sets_the_governor() (gas: 7668)
[PASS] test_constructor_sets_the_timelock() (gas: 8152)
[PASS] test_constructor_sets_the_token() (gas: 8378)
[PASS] test_getLastScheduledUpgradeOperationId_returns_the_operation_id_while_the_upgrade_is_pending() (gas: 2951340)
[PASS] test_getLastScheduledUpgradeOperationId_returns_zero_if_nothing_was_scheduled() (gas: 7750)
[PASS] test_getLastScheduledUpgradeOperationId_still_returns_the_id_after_a_cancel() (gas: 2935577)
[PASS] test_getLastScheduledUpgradeOperationId_still_returns_the_id_after_a_governor_cancel() (gas: 3196116)
[PASS] test_getLastScheduledUpgradeOperationId_still_returns_the_id_after_execution() (gas: 2979202)
[PASS] test_scheduleUpgrade_can_be_rescheduled_after_a_cancel() (gas: 2977199)
[PASS] test_scheduleUpgrade_can_be_rescheduled_after_a_governor_cancel() (gas: 3237323)
[PASS] test_scheduleUpgrade_can_schedule_a_new_upgrade_after_execution() (gas: 5881473)
[PASS] test_scheduleUpgrade_cannot_be_executed_before_the_delay() (gas: 2960063)
[PASS] test_scheduleUpgrade_forwards_arbitrary_upgrade_data() (gas: 2993636)
[PASS] test_scheduleUpgrade_lets_the_council_schedule_a_backup_upgrade() (gas: 2989310)
[PASS] test_scheduleUpgrade_reverts_if_an_upgrade_is_already_pending() (gas: 5809328)
[PASS] test_scheduleUpgrade_reverts_if_the_caller_is_not_the_council() (gas: 2874182)
[PASS] test_scheduleUpgrade_reverts_if_the_implementation_is_the_zero_address() (gas: 9652)
[PASS] test_upgradeDelay_exceeds_the_voter_cancel_cycle() (gas: 28586)
[PASS] test_upgradeDelay_tracks_a_governor_settings_change_through_governance() (gas: 289627)
[PASS] test_upgradeDelay_tracks_a_timelock_minDelay_change() (gas: 32728)
[PASS] test_voter_body_can_cancel_a_council_upgrade_through_the_governor() (gas: 3209970)
[PASS] test_voter_body_can_disarm_the_module_by_revoking_its_roles() (gas: 2933245)
Suite result: ok. 35 passed; 0 failed; 0 skipped; finished in 41.13s (4.84ms CPU time)
```

```
Ran 1 test for test/XanV2.upgrade.e2e.t.sol:XanV2UpgradeE2ETest
[PASS] tableNetworksTest_XanV2_council_scheduling_and_upgrade_succeeds_on_all_supported_networks((string)) (runs: 2, :
↳ 27124040, ~: 27124040)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 55.76s (55.76s CPU time)
```

```
Ran 9 tests for test/XanV2.reinitialization.t.sol:XanV2ReinitializationTest
[PASS] test_delegation_before_reinitialization_does_not_corrupt_the_supply_seed() (gas: 35469525)
[PASS] test_reinitializeFromV1_does_not_change_vote_delegate_votes() (gas: 35398141)
[PASS] test_reinitializeFromV1_emits_the_VestingScheduled_event() (gas: 35392100)
[PASS] test_reinitializeFromV1_reverts_when_called_again() (gas: 16732)
[PASS] test_reinitializeFromV1_reverts_when_the_seed_recipient_already_delegated() (gas: 35369481)
[PASS] test_reinitializeFromV1_reverts_when_the_voting_supply_is_already_seeded() (gas: 35391296)
[PASS] test_reinitializeFromV1_sets_the_owner() (gas: 13253)
[PASS] test_reinitializeFromV1_takes_no_arguments() (gas: 731)
[PASS] test_upgrade_without_reinitialization_grants_no_privileges() (gas: 35399298)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 56.39s (55.94s CPU time)
```

```
Ran 4 tests for test/invariants/InvariantXanV1.t.sol:XanV1Invariants
[PASS] invariant_locked_accounting_holds() (runs: 500, calls: 32000, reverts: 9769)
```

Contract	Selector	Calls	Reverts	Discards
XanV1Handler	airdrop	2508	0	0
XanV1Handler	cancelCouncilUpgrade	2463	1302	0
XanV1Handler	cancelVoterBodyUpgrade	2495	0	0
XanV1Handler	castVote	2414	10	0

XanV1Handler	lock	2443	0	0
XanV1Handler	scheduleCouncilUpgrade	2614	1195	0
XanV1Handler	scheduleVoterBodyUpgrade	2453	2453	0
XanV1Handler	transfer	2461	0	0
XanV1Handler	transferAndLock	2461	2355	0
XanV1Handler	updateStateTracking	2421	0	0
XanV1Handler	updateStateTrackingFor	4813	0	0
XanV1Handler	vetoCouncilUpgrade	2454	2454	0

[PASS] invariant_locked_balance_monotonicity() (runs: 500, calls: 32000, reverts: 9800)

Contract	Selector	Calls	Reverts	Discards
XanV1Handler	airdrop	2339	0	0
XanV1Handler	cancelCouncilUpgrade	2533	1413	0
XanV1Handler	cancelVoterBodyUpgrade	2494	0	0
XanV1Handler	castVote	2442	14	0
XanV1Handler	lock	2517	0	0
XanV1Handler	scheduleCouncilUpgrade	2439	1088	0
XanV1Handler	scheduleVoterBodyUpgrade	2522	2522	0
XanV1Handler	transfer	2497	0	0
XanV1Handler	transferAndLock	2410	2300	0
XanV1Handler	updateStateTracking	2404	0	0
XanV1Handler	updateStateTrackingFor	4939	0	0
XanV1Handler	vetoCouncilUpgrade	2464	2463	0

[PASS] invariant_upgrade_mutual_exclusion() (runs: 500, calls: 32000, reverts: 9618)

Contract	Selector	Calls	Reverts	Discards
XanV1Handler	airdrop	2494	0	0
XanV1Handler	cancelCouncilUpgrade	2492	1372	0
XanV1Handler	cancelVoterBodyUpgrade	2493	8	0
XanV1Handler	castVote	2474	12	0
XanV1Handler	lock	2479	0	0
XanV1Handler	scheduleCouncilUpgrade	2453	1083	0
XanV1Handler	scheduleVoterBodyUpgrade	2386	2384	0
XanV1Handler	transfer	2377	0	0
XanV1Handler	transferAndLock	2518	2388	0
XanV1Handler	updateStateTracking	2435	0	0
XanV1Handler	updateStateTrackingFor	5028	0	0
XanV1Handler	vetoCouncilUpgrade	2371	2371	0

[PASS] invariant_vote_monotonicity() (runs: 500, calls: 32000, reverts: 9839)

Contract	Selector	Calls	Reverts	Discards
XanV1Handler	airdrop	2508	0	0
XanV1Handler	cancelCouncilUpgrade	2497	1453	0
XanV1Handler	cancelVoterBodyUpgrade	2424	0	0
XanV1Handler	castVote	2495	12	0
XanV1Handler	lock	2416	0	0
XanV1Handler	scheduleCouncilUpgrade	2401	1098	0
XanV1Handler	scheduleVoterBodyUpgrade	2436	2435	0
XanV1Handler	transfer	2483	0	0
XanV1Handler	transferAndLock	2480	2359	0
XanV1Handler	updateStateTracking	2532	0	0
XanV1Handler	updateStateTrackingFor	4845	0	0
XanV1Handler	vetoCouncilUpgrade	2483	2482	0

Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 51.33s (125.01s CPU time)

Ran 3 tests for test/invariants/InvariantXanV2.t.sol:XanV2Invariants

[PASS] invariant_locked_balance_accounting() (runs: 500, calls: 32000, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
XanV2Handler	advanceTime	6284	0	0
XanV2Handler	delegate	6355	0	0
XanV2Handler	delegateToSelf	6374	0	0
XanV2Handler	transfer	6399	0	0
XanV2Handler	unlock	6588	0	0

[PASS] invariant_total_supply_constant() (runs: 500, calls: 32000, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
XanV2Handler	advanceTime	6445	0	0
XanV2Handler	delegate	6525	0	0
XanV2Handler	delegateToSelf	6418	0	0
XanV2Handler	transfer	6235	0	0
XanV2Handler	unlock	6377	0	0

[PASS] invariant_votes_equal_delegated_balance() (runs: 500, calls: 32000, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
XanV2Handler	advanceTime	6291	0	0
XanV2Handler	delegate	6443	0	0

XanV2Handler	delegateToSelf	6394	0	0
XanV2Handler	transfer	6433	0	0
XanV2Handler	unlock	6439	0	0

Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 46.91s (86.96s CPU time)

Ran 28 test suites in 68.89s (449.97s CPU time): 231 tests passed, 0 failed, 0 skipped (231 total tests)

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.